Unit 2 Progress Check: MCQ Part B

1. Which of the following code segments can be used to print only the even integers between 2 and 10, inclusive?

Unit 2 Progress Check: MCQ Part B

(A)

```
int num = 0;
while(num < 10)
{
  System.out.println(num);
  num += 2;
}
```

(B)

```
int num = 0;
while(num <= 10)
{
  System.out.println(num);
  num += 2;
}
```

(C) ✓

```
int num = 0;
while(num < 10)
{
  num += 2;
  System.out.println(num);
}
```

Unit 2 Progress Check: MCQ Part B

(D)

```
int num = 0;

while(num <= 10)

{

  num += 2;

  System.out.println(num);

}
```

Answer C

Correct. This code segment initializes num to 0. Inside the loop, num is incremented by 2 and then printed. This continues until num is incremented from 8 to 10, and then the loop terminates.

Unit 2 Progress Check: MCQ Part B

2. In the following code segment, x is an int variable with a positive value.

```
int temp = x;
while (temp > 0)
{
  temp -= 2;
}
System.out.println(temp == 0);
```

Which of the following best describes the behavior of the code segment?

(A) It prints 0 if x is even and prints -1 otherwise.

(B) It prints -1 if x is even and prints 0 otherwise.

(C) It prints true if x is even and prints false otherwise. ✓

(D) It prints false if x is even and prints true otherwise.

Answer C

Correct. The code segment assigns the value of x to temp, then repeatedly subtracts 2 from temp until temp is no longer greater than 0. At this point, temp will be either 0 or -1. If temp is 0, x must be even because it can be obtained by repeatedly adding up the value 2. When temp is 0, the expression temp == 0 evaluates to true.

Unit 2 Progress Check: MCQ Part B

3. Consider the following code segment. Assume that the int variable input has been properly declared and initialized.

```
int answer = 1;
if (input != 0)
{
  int count = 1;

  while (count != input)
  {
    count++;
    answer *= count;
  }
}

System.out.println(answer);
```

Which of the following best describes the condition in which this code segment always results in integer overflow?

(A) When input == 0

(B) When input == 1

(C) When input > 0

(D) When input < 0 ✓

Answer D

Correct. Since count starts as 1 and increases with each iteration, count will never be equal to a negative number. The loop condition will remain true and count will overflow.

Unit 2 Progress Check: MCQ Part B

4. Consider the following code segment.

```
int a = 1;
String result = "";
while (a < 20)
{
  result += a;
  a += 5;
}
System.out.println(result);
```

What is printed as a result of executing this code segment?

(A)

21

(B) ✓

161116

(C)

161161

(D)

16111621

Answer B

Correct. The code segment prints 161116. In the first iteration of the while loop, the string "1" is appended to result and the value of a is set to 6. In the second iteration, the string "6" is appended to result and the value of a is set to 11. In the third iteration, the string "11" is appended to result and the value of a is set to 16. In the fourth (and last) iteration, the string "16" is appended to result, the value of a is set to 21, and the loop terminates because the condition a < 20 is no longer true.

Unit 2 Progress Check: MCQ Part B

5. Consider the following code segment.

```
int sum = 0;
for (int j = 1; /* missing condition */; j += 2)
{
  sum += j;
}
```

Which of the following replacements for /* missing condition */ will cause an infinite loop?

(A)

```
j == 100
```

(B) ✓

```
j != 100
```

(C)

```
j <= 100
```

(D)

```
j >= 100
```

Answer B

Correct. Using this expression, the loop will start with j equal to 1 and will repeatedly increment j by 2. Eventually, the value of j will go from 99 to 101 and never be exactly equal to 100. The loop condition will always be true, leading to an infinite loop.

Unit 2 Progress Check: MCQ Part B

6. Consider the following two code segments, which are intended to produce identical outputs.

Code Segment 1

```
for (int i = 0; i < 10; i++)
{
  System.out.println("counting " + i);
}
```

Code Segment 2

```
int x = 0;
while (x < 10)
{
  x = x + 1;
  System.out.println("counting " + x);
}
```

Which of the following changes can be made to Code Segment 2 so that the outputs of the two code segments are identical as intended?

(A) Move the statement `x = x + 1;` so that it follows the print statement inside the loop body. ✓

(B) Rename the variable `x` to be `i` so that it is the same variable name as in Code Segment 1.

(C) Remove the statement `x = x + 1;` because this statement is not in the loop body in Code Segment 1.

(D) Change the statement `x = x + 1;` to be `x++;` so that the increment of the loop control variable is the same in both code segments.

Answer A

Correct. Moving `x = x + 1;` to the last statement to be executed in the `while` loop body will increment the loop control variable at the same point as the `for` loop in Code Segment 1.

Unit 2 Progress Check: MCQ Part B

7. Consider the following code segment.

```
for (int m = 16; m > 0; m -= 2)
{
  if ((m % 3) == 1)
  {
    System.out.print(m + " ");
  }
}
```

What is printed as a result of executing this code segment?

(A)

15 12 9 6 3

(B)

12 6 0

(C)

16 14 12 10 8 6 4 2

(D) ✓

16 10 4

Answer D

Correct. The code segment initializes m to 16 and decrements m by 2 in each iteration, and prints m only when m % 3 == 1. The loop exits when m is no longer greater than 0.

Unit 2 Progress Check: MCQ Part B

8. In the following code segment, num has been assigned a positive int value. The following code segment is intended to print true if num is even and is intended to print false otherwise.

```
boolean isEven = true;
if (/* missing code */)
{
  isEven = false;
}
System.out.println(isEven);
```

Which of the following can replace /* missing code */ so that this code segment works as intended?

(A)

```
num / 2 == 0
```

(B)

```
num / 2 != 0
```

(C)

```
num % 2 == 0
```

(D) ✓

```
num % 2 != 0
```

Answer D

Correct. This expression uses modulus to check whether an integer is odd. If so, the code segment sets isEven to false. Otherwise, the initial value of isEven, which is true, is unchanged.

Unit 2 Progress Check: MCQ Part B

9. Consider the following code segment.

```
int num = 245;
int temp = num;

while (temp > 0)
{
  System.out.print(temp % 10 + " ");
  temp /= 10;
}
```

What is printed as a result of executing this code segment?

(A)

2 4 5

(B)

4 5

(C)

5 4

(D) ✓

5 4 2

Answer D

Correct. In the while loop, this code segment uses the expression temp % 10 to print the rightmost digit of temp, then discards the rightmost digit by dividing temp by 10 using integer division. Repeating this process will print the digits of num from right to left.

Unit 2 Progress Check: MCQ Part B

10. In the following code segment, assume that low and high are properly declared and initialized int variables and that low < high. The code segment is intended to print the sum of the integers between low and high, inclusive, but does not always work as intended.

```
int sum = 0;         // line 1
int j = low;         // line 2
while (sum <= high)  // line 3
{
  sum += j;          // line 5
  j++;               // line 6
}
System.out.println(sum);
```

Which of the following changes can be made so that this code segment works as intended?

(A) Changing line 1 to int sum = low;

(B) Changing line 2 to int j = 0;

(C) Changing line 3 to while (j <= high) ✓

(D) Interchanging lines 5 and 6

Answer C

Correct. In the given code segment, the loop terminates when sum exceeds high. In order for the code segment to add all the integers from low to high, the loop should continue until j exceeds high.

Unit 2 Progress Check: MCQ Part B

11. In the following code segment, num has been assigned a positive int value.

```
int x = 0;
int temp = num;

while (temp > 0)
{
  x += temp % 10;
  temp /= 10;
}

System.out.println(x);
```

Which of the following best describes the value printed by this code segment?

(A) A count of the number of digits in num

(B) The average of the digits in num

(C) The sum of the digits in num ✓

(D) Whether num is evenly divisible by 10

Answer C

Correct. The code segment uses modulus to separate the digits and adds them within a loop.

Unit 2 Progress Check: MCQ Part B

12. Consider the following code segment.

```
String str = "ABCDEF";
String result = "";
/* missing code */
System.out.println(result);
```

Which of the following can be used to replace `/* missing code */` so that this code segment prints the string `"EFCDAB"`?

Unit 2 Progress Check: MCQ Part B

(A)

```
for (int j = str.length() - 1; j >= 0; j -= 1)
{
  result += str.substring(j, j + 1);
}
```

(B)

```
for (int j = str.length() - 1; j >= 0; j -= 2)
{
  result += str.substring(j, j + 2);
}
```

(C)

```
for (int j = str.length() - 2; j >= 0; j -= 1)
{
  result += str.substring(j, j + 2);
}
```

(D) ✓

```
for (int j = str.length() - 2; j >= 0; j -= 2)
{
  result += str.substring(j, j + 2);
}
```

Answer D

Correct. In the first iteration of the loop, j = 4, causing the loop body to append the substring of str from index 4 to 5 (or "EF") to result. In the second iteration of the loop, j = 2, causing the loop body to append the substring

Unit 2 Progress Check: MCQ Part B

of str from index 2 to 3 (or "CD") to result. In the third and final iteration of the loop, j = 0, causing the loop body to append the substring of str from index 0 to 1 (or "AB") to result.

Unit 2 Progress Check: MCQ Part B

13. In the following code segment, str is a properly declared and initialized string. The code segment is intended to count the number of times that "a" appears in str.

```
int count = 0;
String temp = str;
int loc = temp.indexOf("a");

while (loc >= 0)
{
  /* missing code */
}
```

Which of the following can be used to replace /* missing code */ so that the code segment works as intended?

(A)

```
count++;

temp = str.substring(loc + 1);
```

(B)

```
count++;

loc++;

temp = str.substring(loc);
```

(C)

```
count++;

temp = temp.substring(loc);

loc = temp.indexOf("a");
```

(D) ✓

```
count++;

temp = temp.substring(loc + 1);

loc = temp.indexOf("a");
```

Unit 2 Progress Check: MCQ Part B

Answer D

Correct. Using this code segment, after each `"a"` is found, `count` is incremented by 1, `temp` is assigned the substring starting immediately after the `"a"` that was found, and `loc` is assigned the index of the next `"a"`. When there are no more `"a"`s remaining, `indexOf` will return `-1`, causing the loop to terminate.

Unit 2 Progress Check: MCQ Part B

14. An acronym is formed by extracting the first letter of each word in a phrase of one or more words. For example, the acronym of the phrase "Laugh Out Loud" is "LOL", and the acronym of the phrase "If you know you know" is "Iykyk."

A phrase must meet the following conditions.

- The phrase has exactly one space between each word.
- The phrase does not have leading or trailing spaces.
- The final word in the phrase has at least two characters.

In the following code segment, the String variable phrase has been properly declared and initialized such that it meets these conditions. The code segment is intended to create the acronym of the String stored in phrase and store it in the variable text.

```
String temp = phrase;
String text = temp.substring(0, 1);
int i = temp.indexOf(" ");
while (i > 0)
{
  /* missing code */
}
```

Which of the following can be used to replace /* missing code */ so that the code segment works as intended?

Unit 2 Progress Check: MCQ Part B

(A)

```
text += temp.substring(0, 1);

temp = temp.substring(i + 1);

i = temp.indexOf(" ");
```

(B) ✓

```
temp = temp.substring(i + 1);

text += temp.substring(0, 1);

i = temp.indexOf(" ");
```

(C)

```
text += temp.substring(i, i + 1);

i = i + 1;
```

(D)

```
text += temp.substring(i, i + 1);

i = temp.indexOf(" ");
```

Answer B

Correct. The given code segment initially assigns the value of phrase to temp. The given code segment extracts the first character from the first word in temp and assigns it to text before the while loop. Using this replacement for /* missing code */ inside the loop will correctly extract the first character of the second and subsequent words from temp and concatenate them to text by repeatedly removing the first word from temp, finding the next space, and concatenating the next word's first character to text.

Unit 2 Progress Check: MCQ Part B

15. Consider the following code segment.

```
String str = "qrstu";
String result = "";
for (int j = 0; j < str.length(); j++)
{
  result += str.substring(0, j + 1);
}
System.out.println(result);
```

What, if anything, is printed as a result of executing this code segment?

(A)

rrsrstrstu

(B)

qqrqrsqrst

(C) ✓

qqrqrsqrstqrstu

(D) Nothing is printed because an IndexOutOfBoundsException occurs.

Answer C

Correct. In the first iteration of the loop, j = 0, causing the loop body to append the substring of str from index 0 to 0 (or "q") to result. In the second iteration of the loop, j = 1, causing the loop body to append the substring of str from index 0 to 1 (or "qr") to result. In the third iteration of the loop, j = 2, causing the loop body to append the substring of str from index 0 to 2 (or "qrs") to result. In the fourth iteration of the loop, j = 3, causing the loop body to append the substring of str from index 0 to 3 (or "qrst") to result. In the fifth and last iteration of the loop, j = 4, causing the loop body to append the substring of str from index 0 to 4 (or "qrstu") to result.

Unit 2 Progress Check: MCQ Part B

16. Consider the following code segment.

```
for (int j = 1; j <= 4; j++)        // Line 1
{
  for (int k = j + 1; k >= 1; k--)  // Line 3
  {
    System.out.print(j + " ");      // Line 5
  }
  System.out.println();
}
```

The code segment is intended to produce the following output, but does not work as intended.

```
1
2 2
3 3 3
4 4 4 4
```

Which of the following changes can be made so that the code segment works as intended?

(A) Changing line 1 to `for (int j = 1; j < 4; j++)`

(B) Changing line 3 to `for (int k = 1; k <= j + 1; k++)`

(C) Changing line 3 to `for (int k = j; k >= 1; k--)` ✓

(D) Changing line 5 to `System.out.print(k + " ");`

Answer C

Correct. The given code segment produces the following output because the inner loop iterates one too many times for each iteration of the outer loop.

```
1 1
2 2 2
3 3 3 3
4 4 4 4 4
```

Unit 2 Progress Check: MCQ Part B

Having the inner loop start at `k = j` instead of `k = j + 1` will cause the inner loop to iterate the intended number of times.

17. Consider the following code segment, in which `m` and `n` are properly declared and initialized `int` variables.

```
int result = 0;
for (int i = 0; i < n; i++)
{
  for (int j = m; j < n; j++)
  {
    result++;
  }
}
```

Which of the following best describes the conditions that must be true of `m` and `n` so that after this code segment executes, the value of `result` is greater than 0?

(A) `n > 0` and `m > 0`

(B) `n > 0` and `m > n`

(C) `n > 0` and `n > m` ✓

(D) `m > 0` and `m > n`

Answer C

Correct. If `n` is not greater than 0, the outer loop will never execute. If `n` is not greater than `m`, the inner loop will never execute.

Unit 2 Progress Check: MCQ Part B

18. Consider the following code segment.

```
int a = 1;
while (a <= 2)
{
  int c = 1;
  while (/* missing condition */)
  {
    System.out.print("*");
    c++;
  }
  a++;
}
```

The code segment is intended to print "******".

Which of the following can be used to replace /* missing condition */ so that the code segment works as intended?

(A)

c > 2

(B)

c < 3

(C) ✓

c <= 3

(D)

c >= 3

Answer C

Correct. The outer loop iterates two times (when a is 1 and when a is 2). For six asterisks to be printed, the inner loop needs to iterate three times for each iteration of the outer loop. If the condition of the inner loop is c <= 3, the inner loop will iterate three times.

Unit 2 Progress Check: MCQ Part B

19. Consider the following code segment.

```
double sum = 0.0;
for (int outer = 1; outer <= 4; outer++)
{
   for (int inner = 0; inner <= 4; inner++)
   {
      sum += Math.random();
   }
}
```

How many times will Math.random() be called as a result of executing this code segment?

(A) 5

(B) 9

(C) 16

(D) 20 ✓

Answer D

Correct. The outer loop runs 4 times. For each iteration of the outer loop, the inner loop runs 5 times. Therefore, the Math.random() method is called 4 × 5 times.

Unit 2 Progress Check: MCQ Part B

20. The following code segment appears in a method in the same class as the method doSomething().

```
int i = 1;
while (i < 3)
{
   for (int j = 1; j <= 5; j++)
   {
      doSomething();
   }
   i++;
}
```

How many times will doSomething() be called as a result of executing this code segment?

(A) 5

(B) 8

(C) 10 ✓

(D) 15

Answer C

Correct. The outer loop runs 2 times. For each iteration of the outer loop, the inner loop runs 5 times. Therefore, doSomething() will be called 2 × 5 times.

Unit 2 Progress Check: MCQ Part B

21. Consider the following code segment.

```
for (int x = 1; x <= 3; x++)
{
   for (int y = 5; y > 1; y--)
   {
      System.out.print("*");
   }
}
```

Which of the following best describes the behavior of this code segment?

(A) The print statement is executed 7 times because the outer loop iterates 3 times and the inner loop iterates 4 times.

(B) The print statement is executed 8 times because the outer loop iterates 3 times and the inner loop iterates 5 times.

(C) The print statement is executed 12 times because the outer loop iterates 3 times and the inner loop iterates 4 times for each iteration of the outer loop. ✓

(D) The print statement is executed 15 times because the outer loop iterates 3 times and the inner loop iterates 5 times for each iteration of the outer loop.

Answer C

Correct. The outer loop runs 3 times (when x is 1, 2, and 3). For each iteration of the outer loop, the inner loop runs 4 times (when y is 5, 4, 3, and 2).